INTRODUCTION TO DEEP LEARNING

Course number: 00240332

Lecture 12: Transformers

Xiaolin Hu (胡晓林)

Dept. of Computer Science and Technology

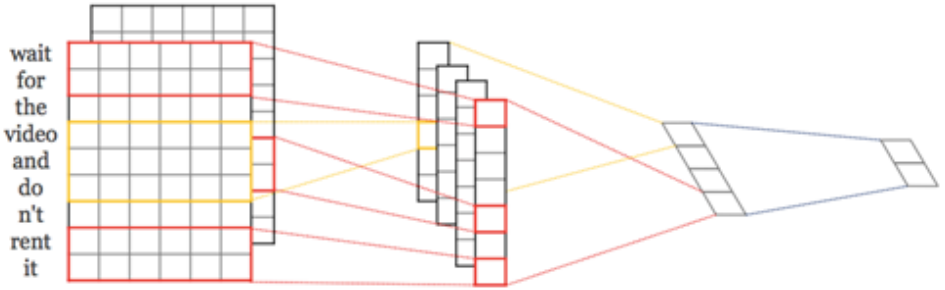
Tsinghua University

Last lecture review

I. Convolutional neural networks

A shallow CNN

A deep CNN

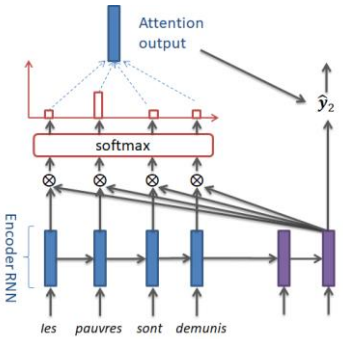


II. Recurrent neural networks

RNN for classification

Seq2seq for NMT

Seq2seq w/ attention



Outline

I. Self-attention ← Modified from John Hewitt's slides for Stanford CS224N
II. Transformer
III. BERT
IV. GPT
V. Summary

Motivation



Beijing has about 20M ... The capital of China

- For RNN, to have interactions between two distant words, it needs O(sequence length) steps
  - The memory of RNN may not be so long
  - It cannot computed in parallel
- For CNN, you need to go to deeper layers, but the representations of the words has changed a lot, which may not be what you want
- MLP is able to catch the information of such two words in every layer, but it cannot handle *variable length* input

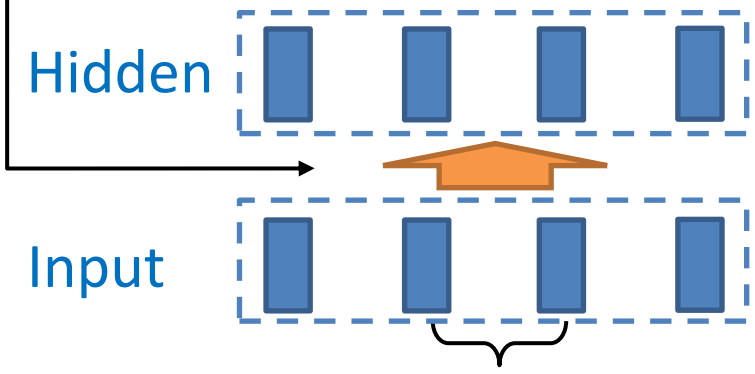
Attention comes to rescue !

An ideal model

It has interactions between any pair of units in single layers

Information can be processed in parallel

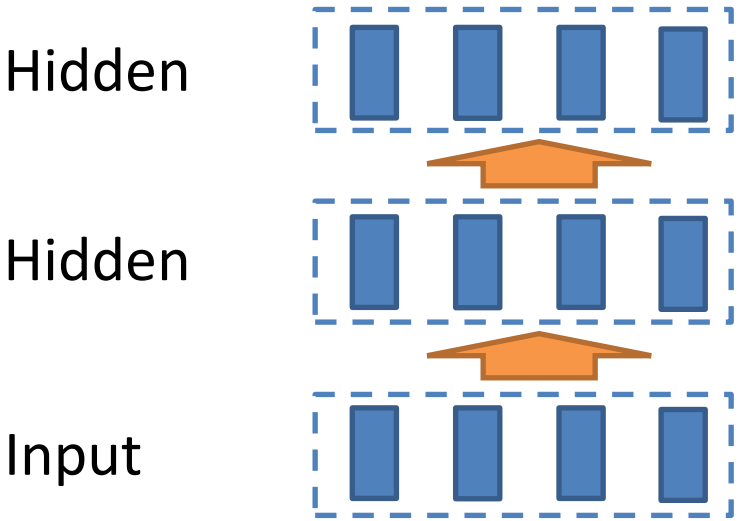
It is able to handle variable length input



1. Let the number of hidden units depend on the number of inputs
2. Hidden units share weights

How to perform tasks with variable length input?

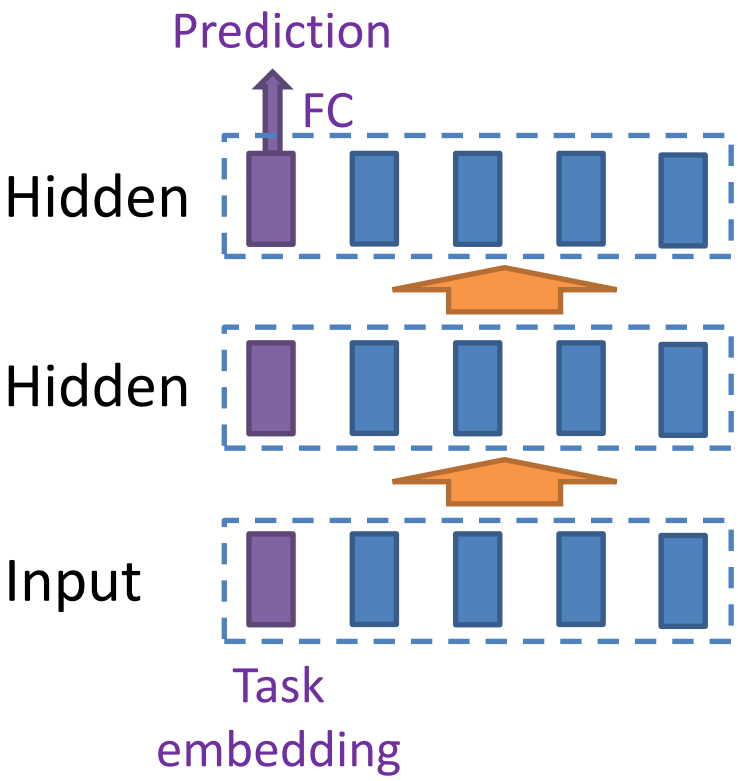
Key observation: Every unit has the information of all other units in the same layer

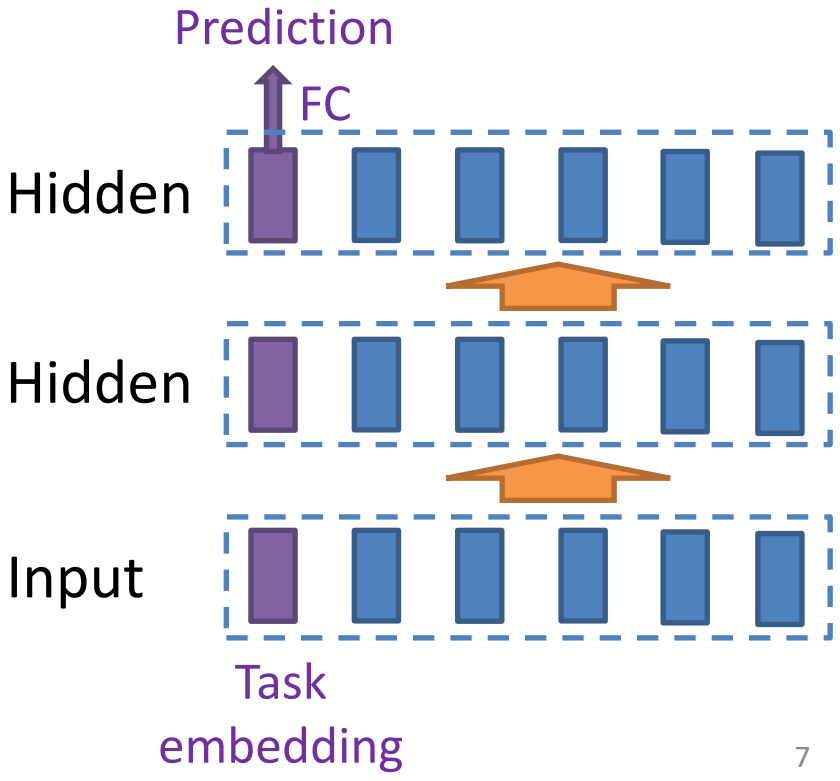


How to perform tasks with variable length input?

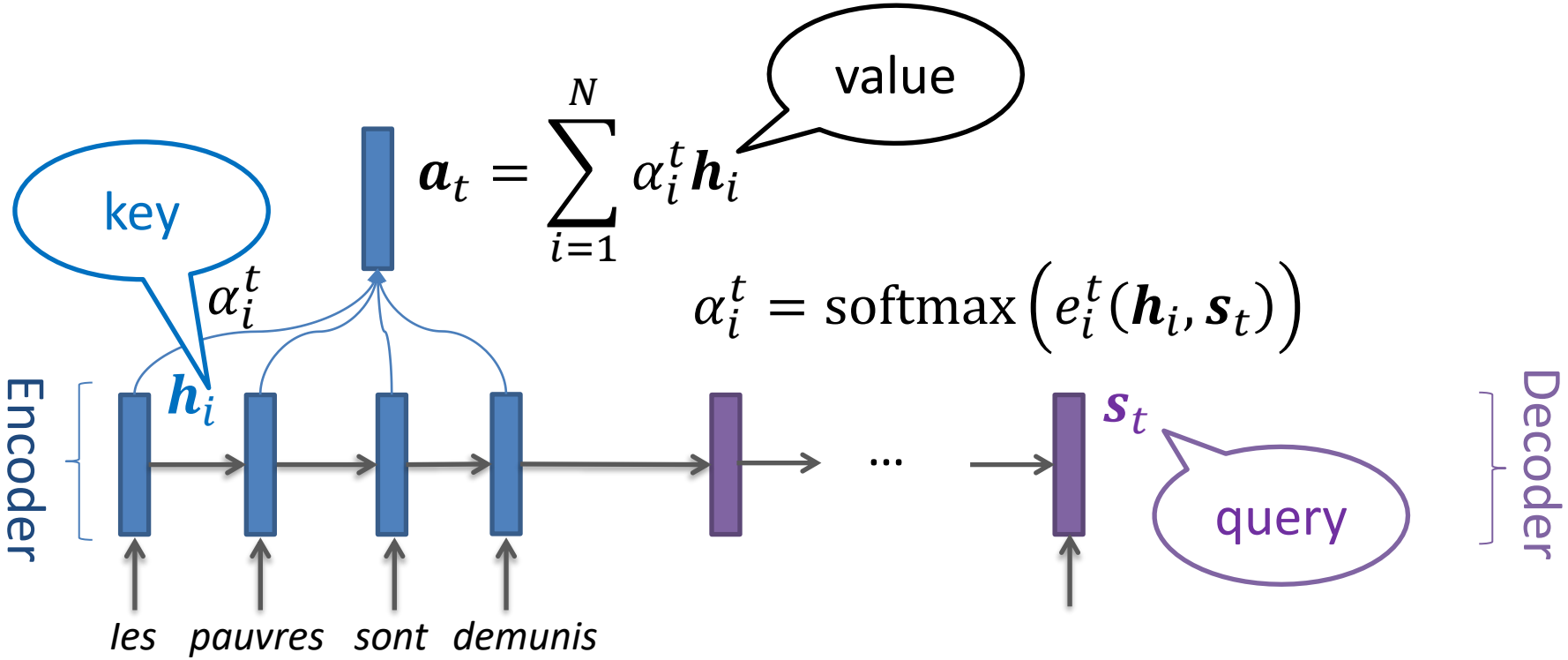
Key observation: Every unit has the information of all other units in the same layer

Strategy: Add a task-related token in every layer





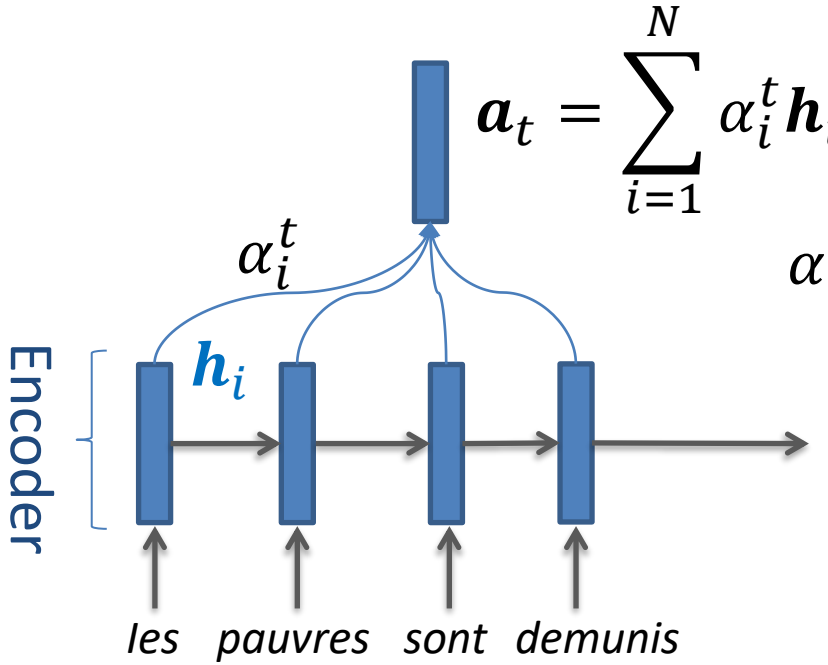
Recap: Cross attention



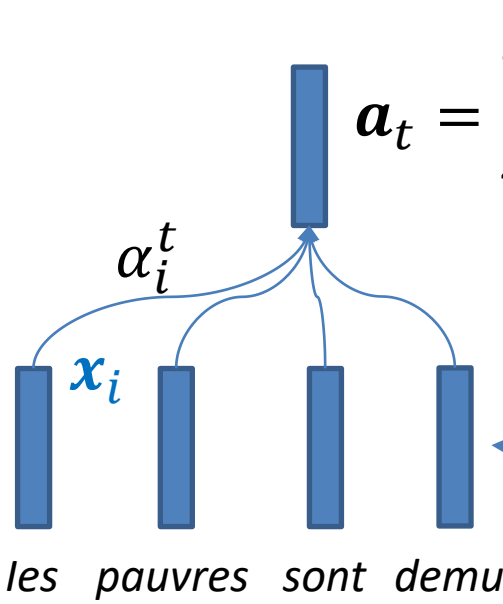
$$\boldsymbol{a}_t = \sum_{i=1}^{N} \mathrm{softmax}\left(e_i^t(\boldsymbol{h}_i, \boldsymbol{s}_t)\right) \boldsymbol{h}_i$$

key query value

Step 1: remove the decoder



Step 2: remove the RNN



$$\boldsymbol{a}_t = \sum_{i=1}^{N} \alpha_i^t \boldsymbol{x}_i$$

The number of $\boldsymbol{a}_t$ is determined by the number of $\boldsymbol{s}_t$

$$\alpha_i^t = \mathrm{softmax}\left(e_i^t(\boldsymbol{x}_i, \boldsymbol{s}_t)\right)$$

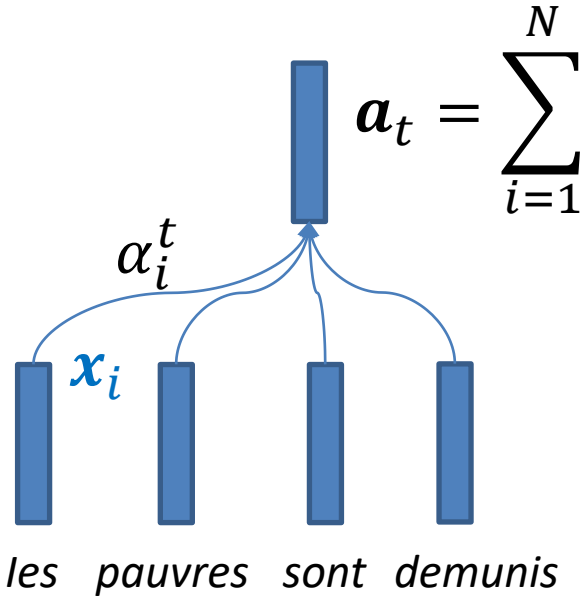
Word embeddings instead of hidden states of RNN

Time dependence is removed!

If we could decide α_i^t from the input words *automatically*, no matter what N is, we have one $\boldsymbol{a}_t$

- We can predefine the number of outputs $\boldsymbol{a}_t$'s

Step 2: remove the RNN



How to determine the weights α_i^t?

$$\alpha_i^t = \text{softmax}\left(e_i^t(\boldsymbol{x}_i, \boldsymbol{s}_t)\right)$$

key query

- The output $\boldsymbol{a}_t$ has info of all words!
- We calculate interactions between every pair of units

A simple idea is to use $\boldsymbol{x}_t$ as the query, then $\{\alpha_i^t\}_{i=1\ldots N;t=1\ldots N}$ is a square matrix

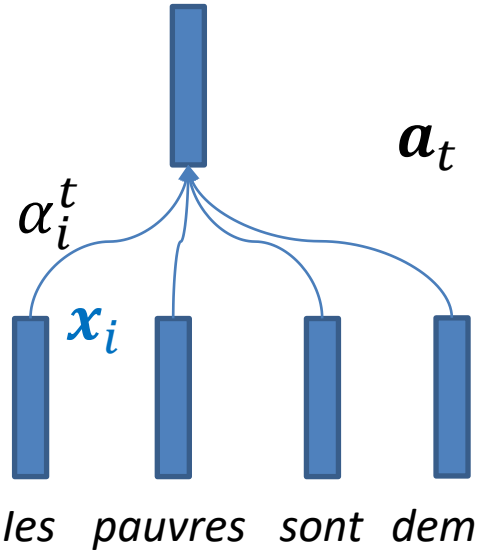
The number of $\boldsymbol{a}_t$'s becomes N

Step 3: introduce parameters

Vaswani et al., 2017

Input: $\boldsymbol{x}_i$ output: $\boldsymbol{a}_t$

No parameters to learn!



$$\boldsymbol{a}_t = \sum_{i=1}^{N} \alpha_i^t \boldsymbol{x}_i = \sum_{i=1}^{N} \text{softmax}\left(e_i^t(\boldsymbol{x}_i, \boldsymbol{x}_t)\right) \boldsymbol{x}_i$$

key query value

$$\boldsymbol{a}_t = \sum_{i=1}^{N} \text{softmax}\left(e_i^t(\boldsymbol{k}_i, \boldsymbol{q}_t)\right) \boldsymbol{v}_i$$

Parameterize the keys, queries and values: $\boldsymbol{x}_i \in R^{1\times d}$

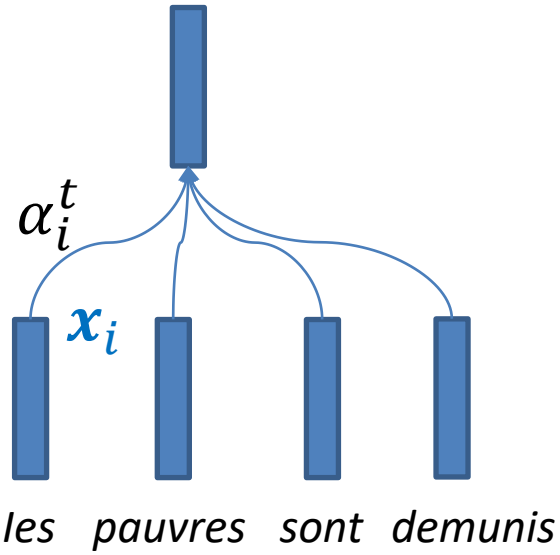
$$\boldsymbol{x}_i \xrightarrow{\boldsymbol{W}^{\mathrm{K}}} \boldsymbol{k}_i = \boldsymbol{x}_i\boldsymbol{W}^{\mathrm{K}} \qquad \boldsymbol{W}^{\mathrm{K}} \in R^{d\times d_{\mathrm{K}}} \qquad \boldsymbol{k}_i \in R^{1\times d_{\mathrm{K}}}$$

$$\boldsymbol{x}_i \xrightarrow{\boldsymbol{W}^{\mathrm{Q}}} \boldsymbol{q}_i = \boldsymbol{x}_i\boldsymbol{W}^{\mathrm{Q}} \qquad \boldsymbol{W}^{\mathrm{Q}} \in R^{d\times d_{\mathrm{Q}}} \qquad \boldsymbol{q}_i \in R^{1\times d_{\mathrm{Q}}}$$

$$\boldsymbol{x}_i \xrightarrow{\boldsymbol{W}^{\mathrm{V}}} \boldsymbol{v}_i = \boldsymbol{x}_i\boldsymbol{W}^{\mathrm{V}} \qquad \boldsymbol{W}^{\mathrm{V}} \in R^{d\times d_{\mathrm{V}}} \qquad \boldsymbol{v}_i \in R^{1\times d_{\mathrm{V}}}$$

Step 3: introduce parameters

Vaswani et al., 2017



The variable length input problem is solved!

$\boldsymbol{W}^{\mathrm{K}}$, $\boldsymbol{W}^{\mathrm{Q}}$ and $\boldsymbol{W}^{\mathrm{V}}$ are the same for all $\boldsymbol{x}_i$

Parameterize the keys, queries and values: $\boldsymbol{x}_i \in R^{1\times d}$

$\boldsymbol{x}_i$ → $\boldsymbol{W}^{\mathrm{K}}$: $\boldsymbol{k}_i = \boldsymbol{x}_i \boldsymbol{W}^{\mathrm{K}}$, $\boldsymbol{W}^{\mathrm{K}} \in R^{d\times d_{\mathrm{K}}}$, $\boldsymbol{k}_i \in R^{1\times d_{\mathrm{K}}}$

$\boldsymbol{x}_i$ → $\boldsymbol{W}^{\mathrm{Q}}$: $\boldsymbol{q}_i = \boldsymbol{x}_i \boldsymbol{W}^{\mathrm{Q}}$, $\boldsymbol{W}^{\mathrm{Q}} \in R^{d\times d_{\mathrm{Q}}}$, $\boldsymbol{q}_i \in R^{1\times d_{\mathrm{Q}}}$

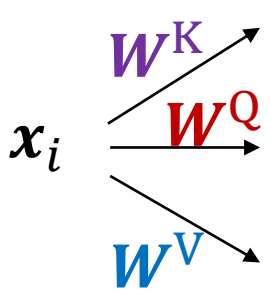
$\boldsymbol{x}_i$ → $\boldsymbol{W}^{\mathrm{V}}$: $\boldsymbol{v}_i = \boldsymbol{x}_i \boldsymbol{W}^{\mathrm{V}}$, $\boldsymbol{W}^{\mathrm{V}} \in R^{d\times d_{\mathrm{V}}}$, $\boldsymbol{v}_i \in R^{1\times d_{\mathrm{V}}}$

Self-attention: formal formulation

Vaswani et al., 2017

$$\boldsymbol{a}_t = \sum_{i=1}^{N} \text{softmax}\left(e_i^t(\boldsymbol{k}_i, \boldsymbol{q}_t)\right) \boldsymbol{v}_i \quad \in R^{1\times d_\text{V}}$$

Parameterize the keys, queries and values: $\boldsymbol{x}_i \in R^{1\times d}$



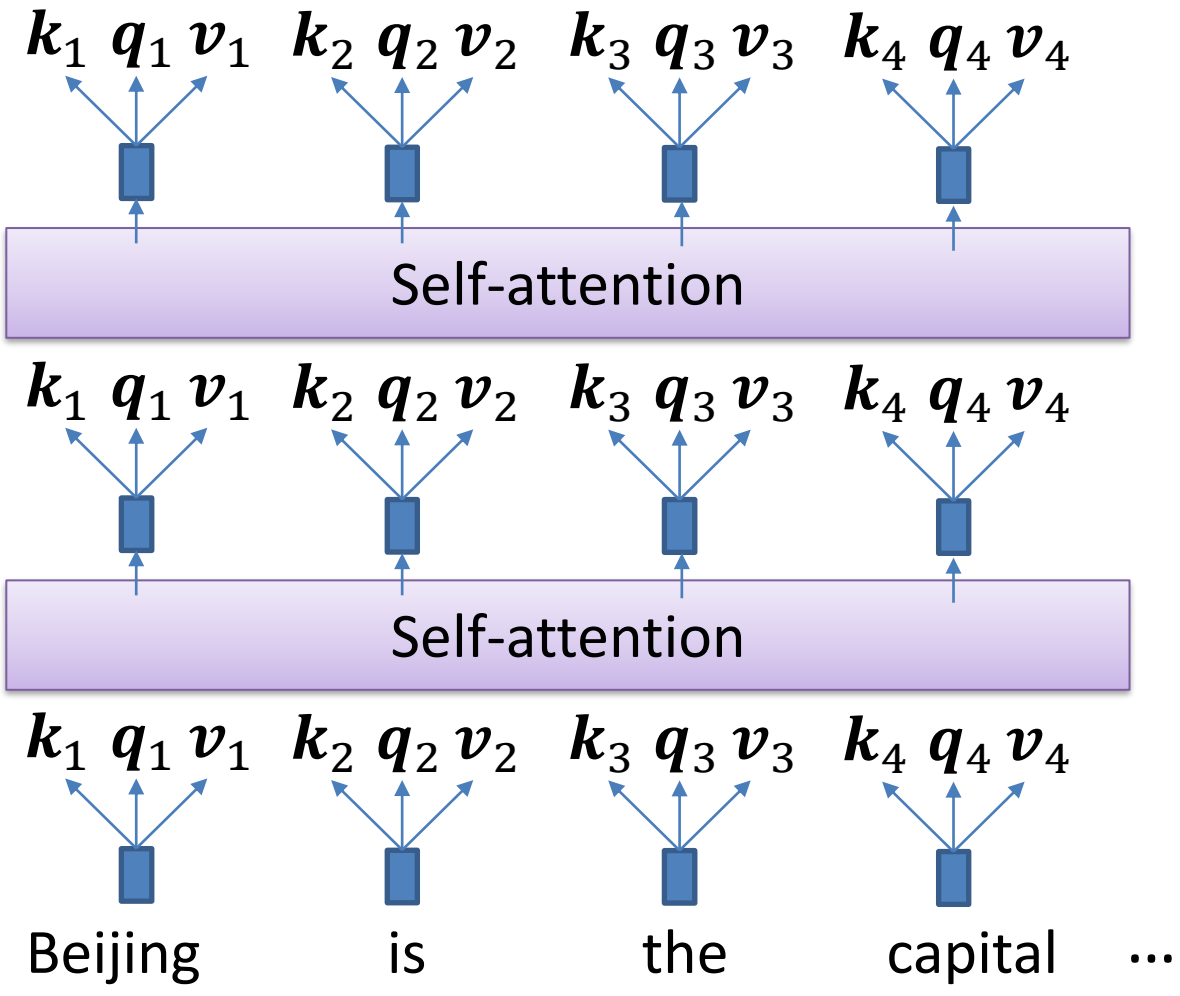
$\boldsymbol{k}_i = \boldsymbol{x}_i\boldsymbol{W}^\text{K}$ $\quad \boldsymbol{W}^\text{K} \in R^{d\times d_\text{K}}$ $\quad \boldsymbol{k}_i \in R^{1\times d_\text{K}}$

$\boldsymbol{q}_i = \boldsymbol{x}_i\boldsymbol{W}^\text{Q}$ $\quad \boldsymbol{W}^\text{Q} \in R^{d\times d_\text{Q}}$ $\quad \boldsymbol{q}_i \in R^{1\times d_\text{Q}}$

$\boldsymbol{v}_i = \boldsymbol{x}_i\boldsymbol{W}^\text{V}$ $\quad \boldsymbol{W}^\text{V} \in R^{d\times d_\text{V}}$ $\quad \boldsymbol{v}_i \in R^{1\times d_\text{V}}$

- If you use dot product *(often the case)* for e_i^t, then $d_\text{Q} = d_\text{K}$
- $\boldsymbol{W}^\text{V}$ is used to set the dimension of the output $\boldsymbol{a}_t$

Self-attention as a building block



- No elementwise nonlinearities

  Introduce here

- It does not know the order of the input

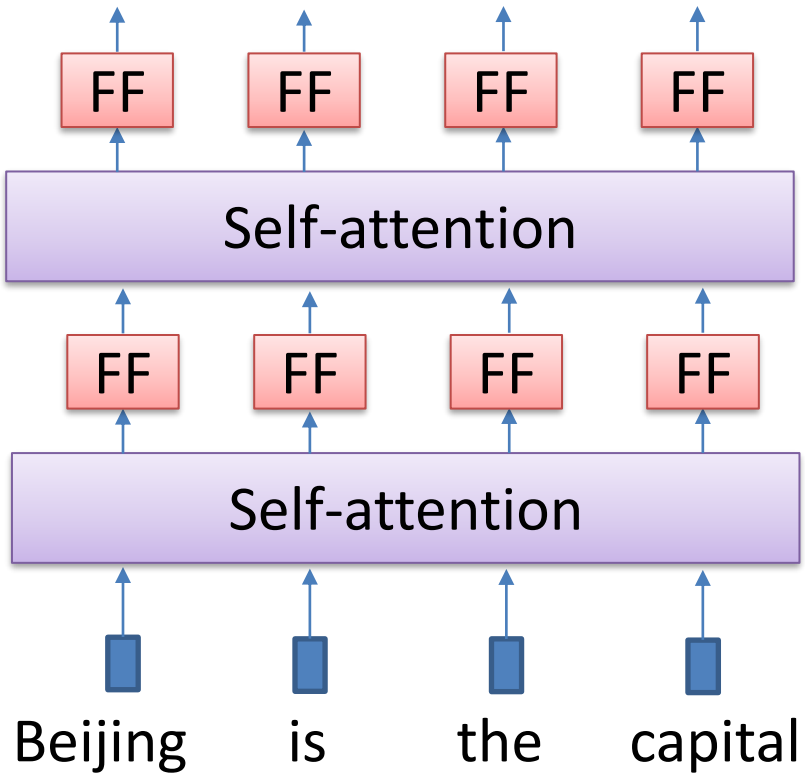
  Position encoding (see Stanford CS224N)

Add elementwise nonlinearity

- No elementwise non-linearities in self-attention
- Stacking more self-attention layers just re-averages value vectors

Add a feed-forward network to post-process each output vector

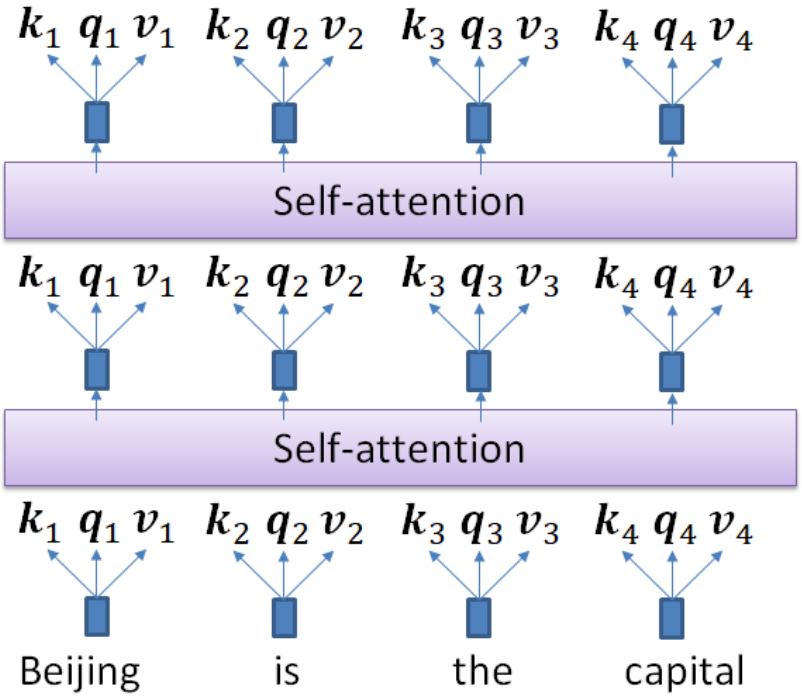
$$\boldsymbol{m}_t = \mathrm{MLP}(\boldsymbol{a}_t)$$
$$= \boldsymbol{W}_2 \mathrm{ReLU}(\boldsymbol{W}_1 \boldsymbol{a}_t + \boldsymbol{b}_1) + \boldsymbol{b}_2$$

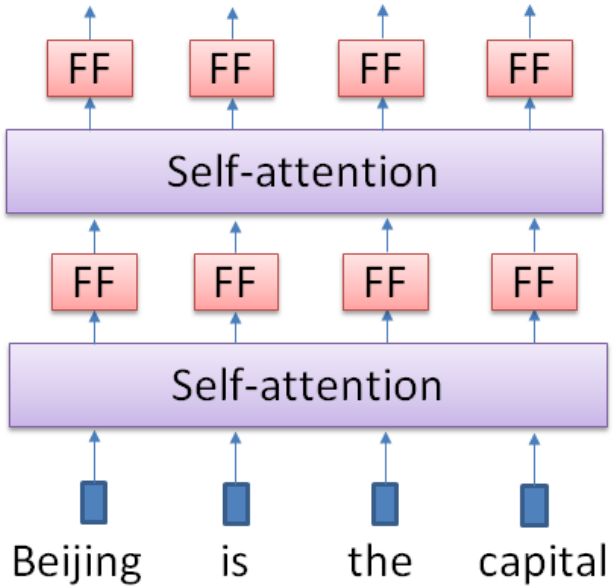


Summary of Part I

Input x_i $\xrightarrow{W^{\mathrm{K}}}$ k_i, $\xrightarrow{W^{\mathrm{Q}}}$ q_i, $\xrightarrow{W^{\mathrm{V}}}$ v_i

Output $a_t = \sum_{i=1}^{N} \mathrm{softmax}\left(e_i^t(k_i, q_t)\right) v_i$





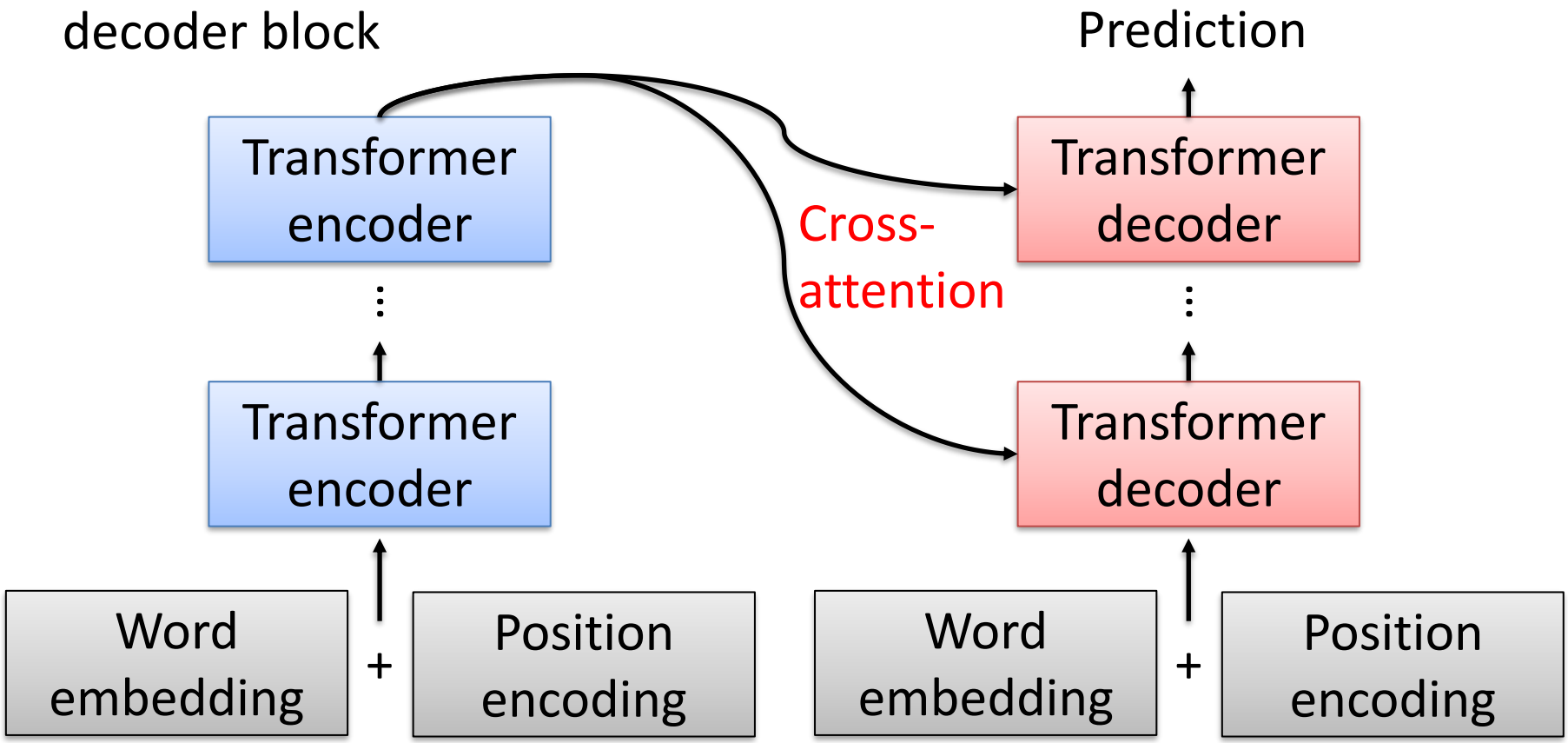
Outline

I. Self-attention
II. Transformer
III. BERT
IV. GPT
V. Summary

Transformer

Vaswani et al., 2017

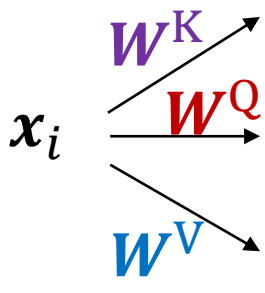
Self-attention in each encoder and decoder block



Self-attention: matrix-vector form

$$\boldsymbol{a}_t = \sum_{i=1}^{N} \text{softmax}\left(e_i^t(\boldsymbol{k}_i, \boldsymbol{q}_t)\right) \boldsymbol{v}_i \in R^{1\times d_\text{V}}$$

Parameterize the keys, queries and values: $\boldsymbol{x}_i \in R^{1\times d}$



$\boldsymbol{k}_i = \boldsymbol{x}_i\boldsymbol{W}^\text{K}$ $\quad \boldsymbol{W}^\text{K} \in R^{d\times d_\text{K}}$ $\quad \boldsymbol{k}_i \in R^{1\times d_\text{K}}$

$\boldsymbol{q}_i = \boldsymbol{x}_i\boldsymbol{W}^\text{Q}$ $\quad \boldsymbol{W}^\text{Q} \in R^{d\times d_\text{Q}}$ $\quad \boldsymbol{q}_i \in R^{1\times d_\text{Q}}$

$\boldsymbol{v}_i = \boldsymbol{x}_i\boldsymbol{W}^\text{V}$ $\quad \boldsymbol{W}^\text{V} \in R^{d\times d_\text{V}}$ $\quad \boldsymbol{v}_i \in R^{1\times d_\text{V}}$

- Let's use dot product for e_i^t, then $d_\text{Q} = d_\text{K}$
- Let $\boldsymbol{X} = (\boldsymbol{x}_1; \ldots; \boldsymbol{x}_N) \in R^{N\times d}$
- Then $\boldsymbol{K} = \boldsymbol{X}\boldsymbol{W}^\text{K} \in R^{N\times d_\text{K}}, \boldsymbol{Q} = \boldsymbol{X}\boldsymbol{W}^\text{Q} \in R^{N\times d_\text{K}}, \boldsymbol{V} = \boldsymbol{X}\boldsymbol{W}^\text{V} \in R^{N\times d_\text{V}}$

Self-attention: matrix-vector form

$\boldsymbol{X} = (\boldsymbol{x}_1; \ldots; \boldsymbol{x}_N) \in R^{N \times d}$,
$\boldsymbol{K} = \boldsymbol{X}\boldsymbol{W}^{\mathrm{K}} \in R^{N \times d_{\mathrm{K}}}, \boldsymbol{Q} = \boldsymbol{X}\boldsymbol{W}^{\mathrm{Q}} \in R^{N \times d_{\mathrm{K}}}, \boldsymbol{V} = \boldsymbol{X}\boldsymbol{W}^{\mathrm{V}} \in R^{N \times d_{\mathrm{V}}}$

Let $\boldsymbol{A} = (\boldsymbol{a}_1; \ldots; \boldsymbol{a}_N) \in R^{N \times d_{\mathrm{V}}}$, where

$$\boldsymbol{a}_t = \sum_{i=1}^{N} \mathrm{softmax}\left(e_i^t(\boldsymbol{k}_i, \boldsymbol{q}_t)\right) \boldsymbol{v}_i$$

then

$$\boldsymbol{A} = \mathrm{softmax}(\boldsymbol{Q}\boldsymbol{K}^{\top})\boldsymbol{V}$$

- $\boldsymbol{Q}\boldsymbol{K}^{\top} \in R^{N \times N}$
- Softmax is applied in each row
- The t-th row of Softmax() contains α_i^t $(i = 1, \ldots, N)$ to weight the rows of $\boldsymbol{V}$

Multi-headed attention

- Increase the capacity of the model
  - For each input $\boldsymbol{X}$ we introduce multiple matrices $\boldsymbol{W}^{\mathrm{K}}$, $\boldsymbol{W}^{\mathrm{Q}}$ and $\boldsymbol{W}^{\mathrm{V}}$
- Suppose there are h heads, for $l \in \{1, \ldots, h\}$

Single-head	Multi-head
$\boldsymbol{W}^{\mathrm{K}} \in R^{d \times d_{\mathrm{K}}}$	$\boldsymbol{W}_l^{\mathrm{K}} \in R^{d \times \frac{d_{\mathrm{K}}}{h}}$
$\boldsymbol{W}^{\mathrm{Q}} \in R^{d \times d_{\mathrm{K}}}$	$\boldsymbol{W}_l^{\mathrm{Q}} \in R^{d \times \frac{d_{\mathrm{K}}}{h}}$
$\boldsymbol{W}^{\mathrm{V}} \in R^{d \times d_{\mathrm{V}}}$	$\boldsymbol{W}_l^{\mathrm{V}} \in R^{d \times \frac{d_{\mathrm{V}}}{h}}$

$$\boldsymbol{K}_l = \boldsymbol{X}\boldsymbol{W}_l^{\mathrm{K}} \in R^{N \times \frac{d_{\mathrm{K}}}{h}}$$

$$\boldsymbol{Q}_l = \boldsymbol{X}\boldsymbol{W}_l^{\mathrm{Q}} \in R^{N \times \frac{d_{\mathrm{K}}}{h}}$$

$$\boldsymbol{V}_l = \boldsymbol{X}\boldsymbol{W}_l^{\mathrm{V}} \in R^{N \times \frac{d_{\mathrm{V}}}{h}}$$

(The vectors of keys, queries and values are shorter)

- Each attention head performs attention independently

$$\boldsymbol{A}_l = \mathrm{softmax}\left(\boldsymbol{Q}_l \boldsymbol{K}_l^{\top}\right)\boldsymbol{V}_l \in R^{N \times d_{\mathrm{V}}/h}$$

- The outputs of all the heads are combined!

$$\boldsymbol{A} = (\boldsymbol{A}_1; \ldots; \boldsymbol{A}_h) \in R^{N \times d_{\mathrm{V}}}$$ ← Same as before

Scaled dot product

- For each head

$$\boldsymbol{A}_l = \mathrm{softmax}\left(\boldsymbol{Q}_l \boldsymbol{K}_l^{\top}\right)\boldsymbol{V}_l$$

$$\boldsymbol{Q}_l = \boldsymbol{X}\boldsymbol{W}_l^{\mathrm{Q}} \in R^{N \times \frac{d_{\mathrm{K}}}{h}}$$

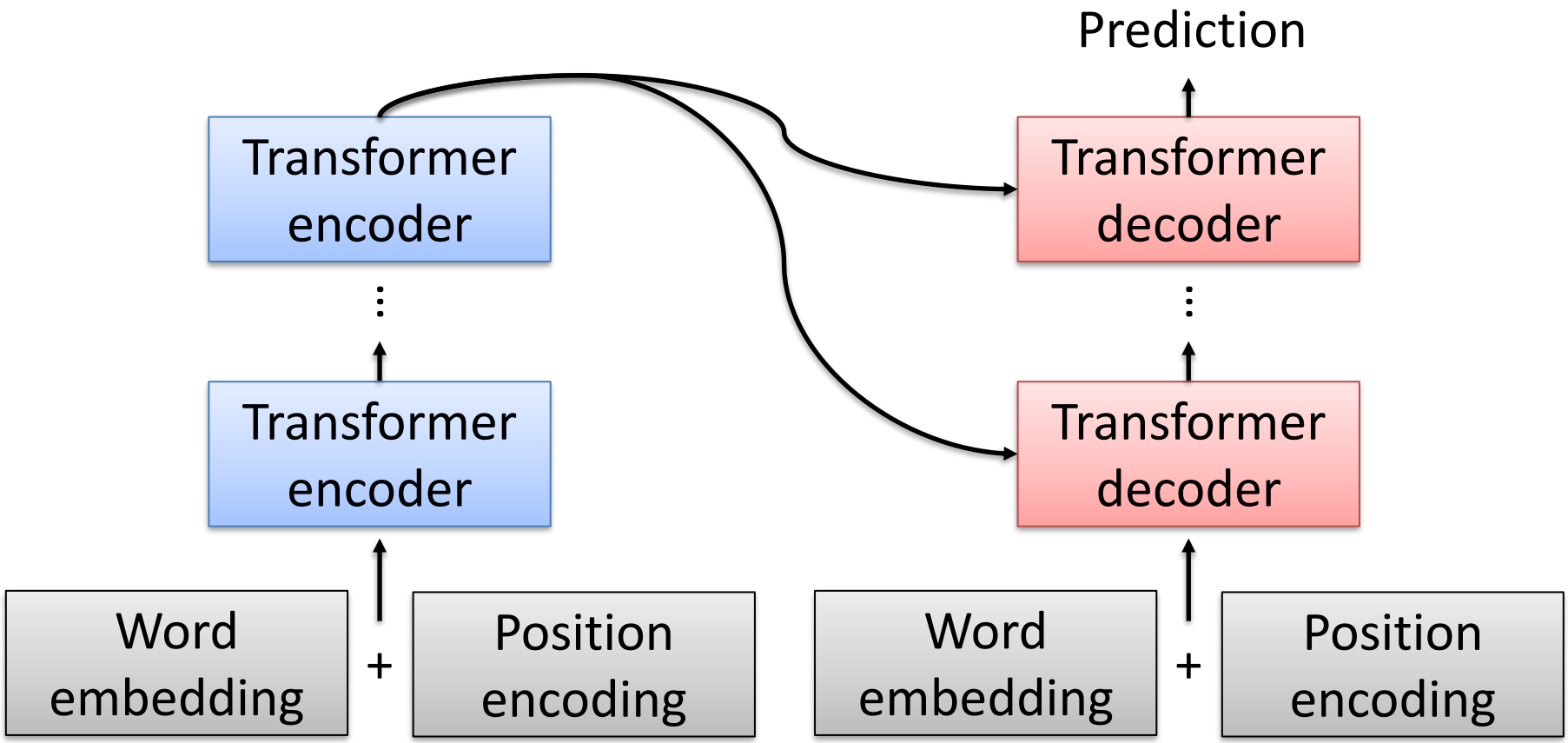
$$\boldsymbol{K}_l = \boldsymbol{X}\boldsymbol{W}_l^{\mathrm{K}} \in R^{N \times \frac{d_{\mathrm{K}}}{h}}$$

- When d_{K}/h is large, dot products between vectors tend to become large
  - Inputs to the softmax function can be large, making the gradients small
- In practice, we use the scaled dot product

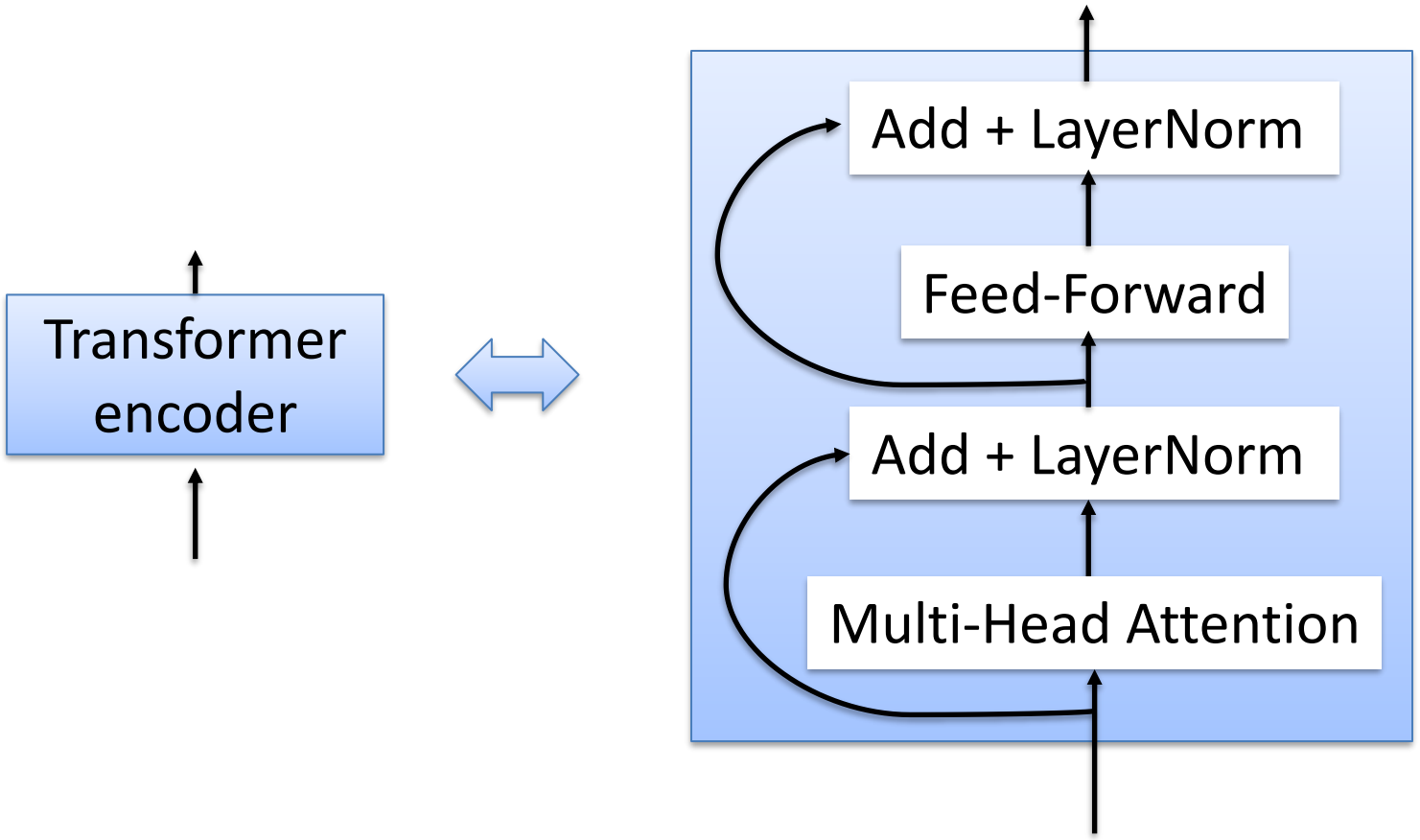
$$\boldsymbol{A}_l = \mathrm{softmax}\left(\frac{\boldsymbol{Q}_l \boldsymbol{K}_l^{\top}}{\sqrt{d_{\mathrm{K}}/h}}\right)\boldsymbol{V}_l$$

Transformer

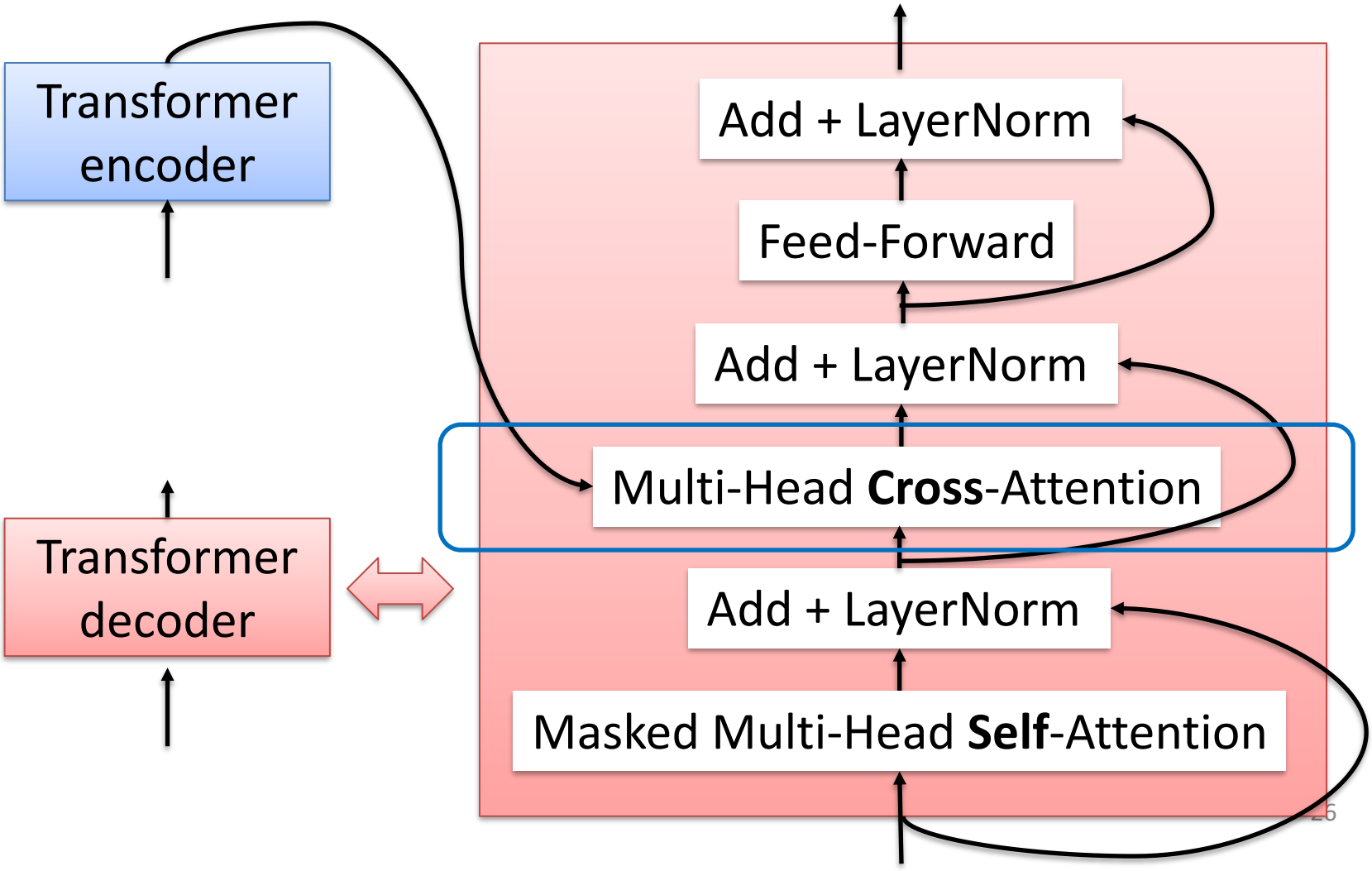
Vaswani et al., 2017



In a Transformer encoder

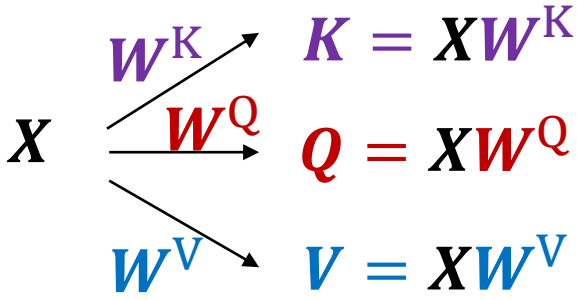


In a Transformer decoder

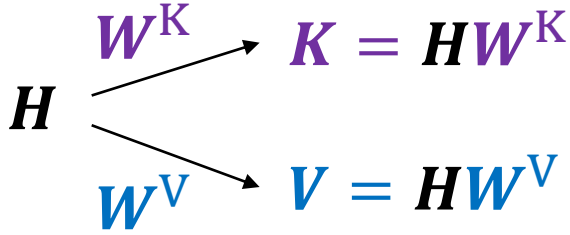


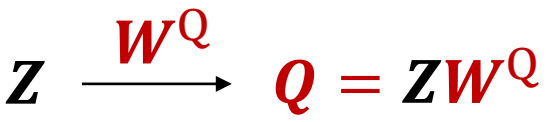
Cross-attention

- In self-attention, the keys, queries and values come from the same source $\boldsymbol{X}$



- In cross-attention
  - The keys and values come from the encoder's output $\boldsymbol{H}$
  - The queries come from the decoder's input $\boldsymbol{Z}$





Other operations are the same as in the self-attention

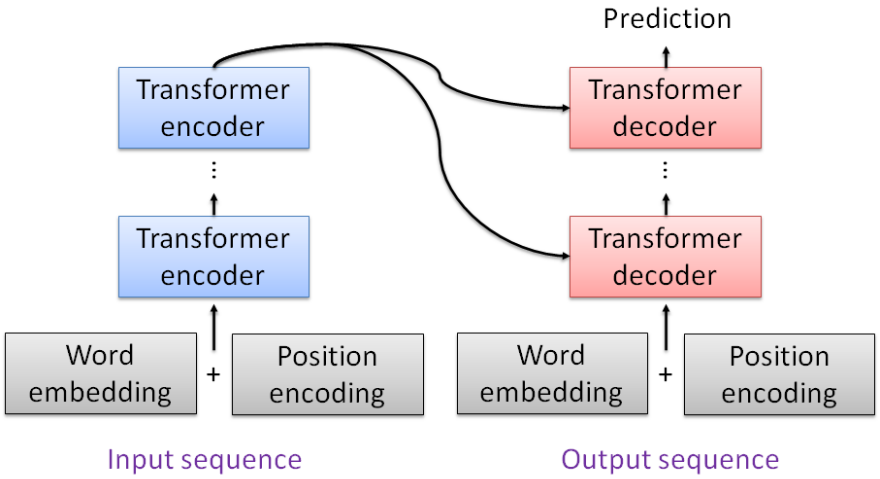
All techniques in Transformer

- We have covered
  - key-query-value
  - Multi-headed attention
  - Scaling the dot product

- We haven't covered
  - Position encoding
  - Layer normalization
  - Masking the future in attention

Please read Stanford CS224N slides

Encoder VS decoder



Encoder

1. It can attend to all tokens
2. Its output has the same number of tokens as input
3. It has self-attention only

Decoder

1. It can only attend to some tokens
2. Its output is just one token
3. It has both self- and cross-attention

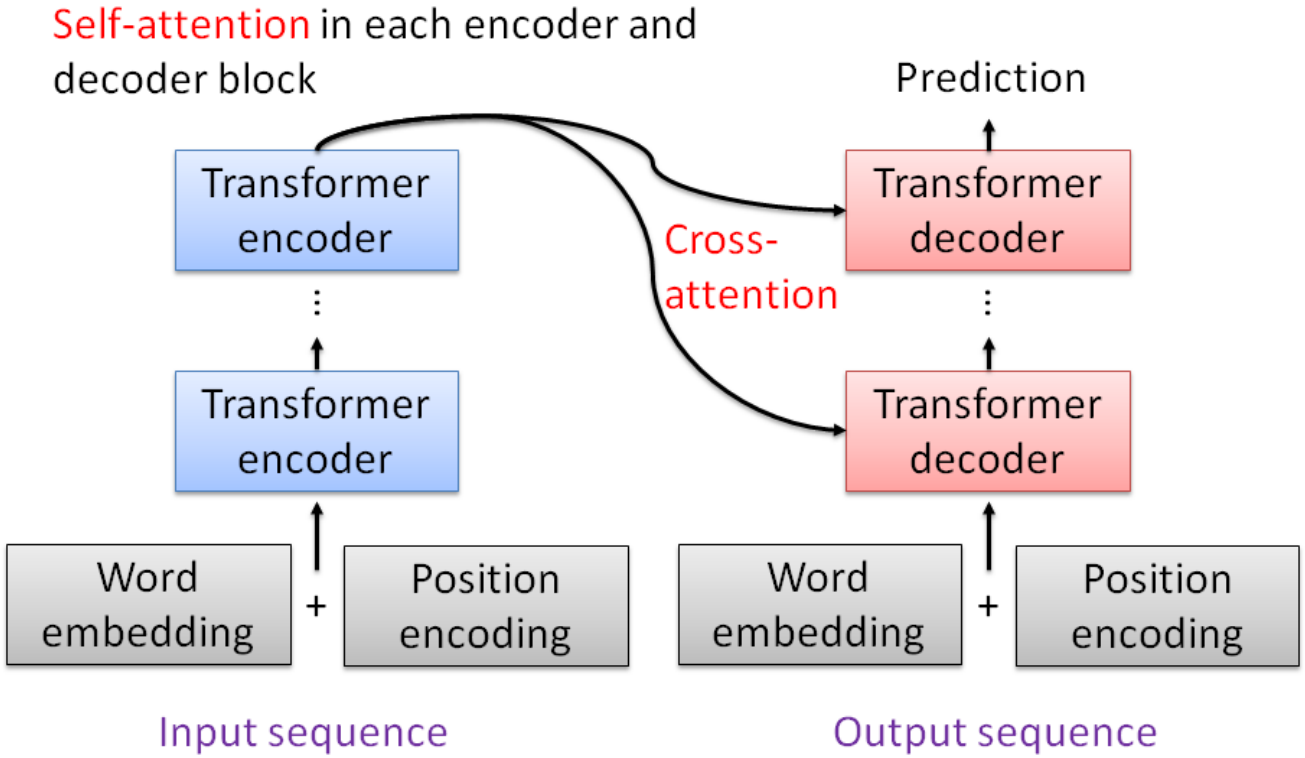
Results on NMT

Vaswani et al., 2017

English German French

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [15]	23.75			
Deep-Att + PosUnk [32]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [31]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [8]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [26]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [32]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [31]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [8]	26.36	**41.29**	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	**$3.3 \cdot 10^{18}$**	
Transformer (big)	**28.4**	**41.0**	$2.3 \cdot 10^{19}$	

Summary of Part II



$$\boldsymbol{A}_l = \text{softmax}\left(\frac{\boldsymbol{Q}_l \boldsymbol{K}_l^\top}{\sqrt{d_\text{K}/h}}\right)\boldsymbol{V}_l$$

- In self-attention, $\boldsymbol{Q}_l, \boldsymbol{K}_l, \boldsymbol{V}_l$ are from the encoder
- In cross-attention, $\boldsymbol{Q}_l$ is from the decoder

单选题 1分

设置

Suppose we have trained a model with self-attention layers. It uses more parameters to handle longer input sequence.

A Yes

B No

提交

多选题 1分

设置

For the cross-attention of Transformer’s decoder, which of the following are true?

A The queries are from the decoder

B The values are from the decoder

C The keys are from the encoder

D The queries are from the encoder

提交

Outline

I. Self-attention
II. Transformer
III. BERT
IV. GPT
V. Summary

Pre-training and fine-tuning

- When task-related data is insufficient, we take:



(On a large dataset)

- Computer vision example
  - Pre-train a CNN on ImageNet, then fine-tune it on a small image dataset
  - The tasks for pre-training and fine-tuning can be different
  - Pre-training can be supervised or unsupervised

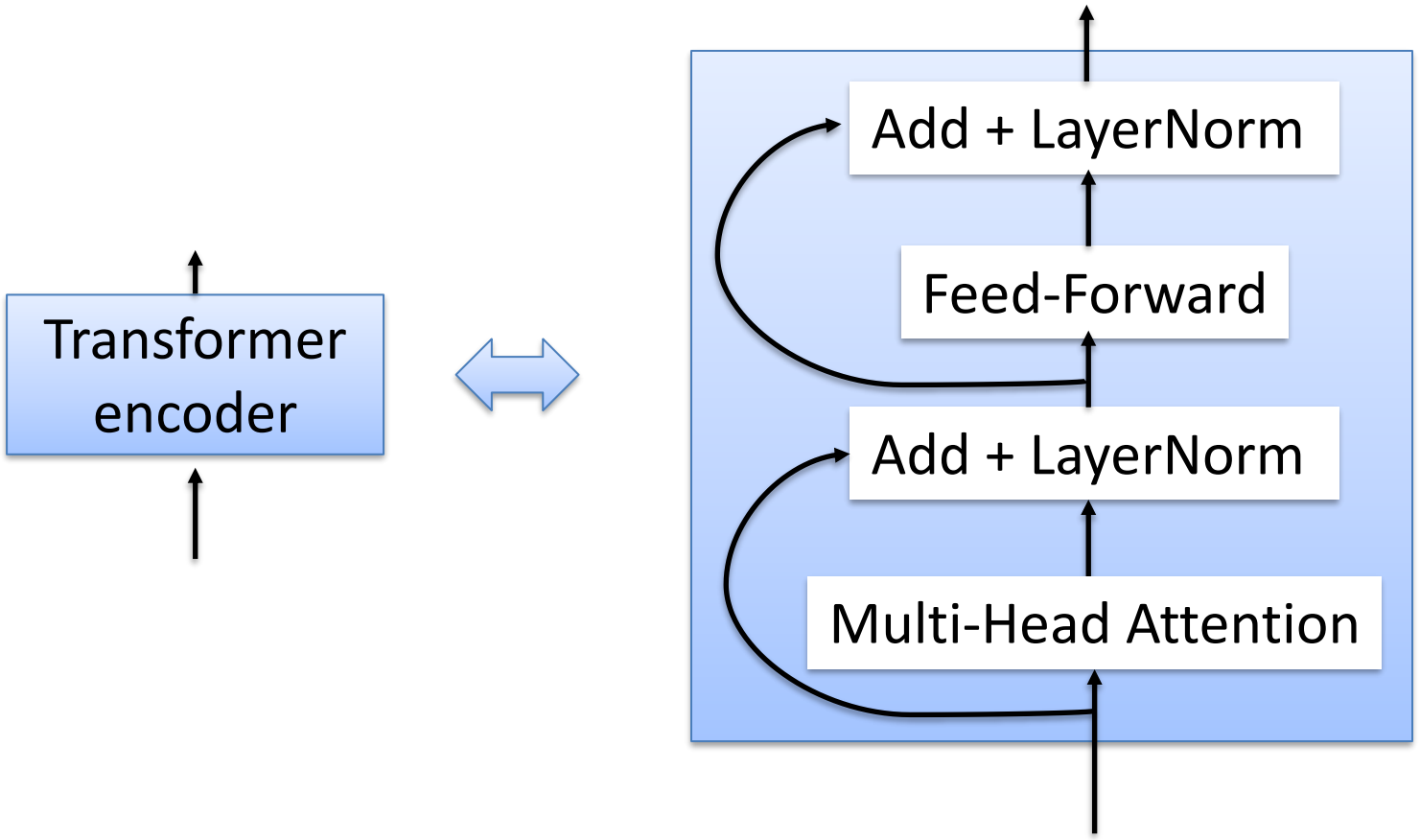
Now we consider the two-step approach for NLP

Pre-training methods

- Word2Vec, Glove, etc.
- Embeddings from Language Models (ELMo)
  - Based on bi-directional LSTM

- Bidirectional Encoder Representations from Transformers (BERT)
  - Based on Transformer encoder
- Generative Pre-training (GPT)
  - Based on Transformer decoder

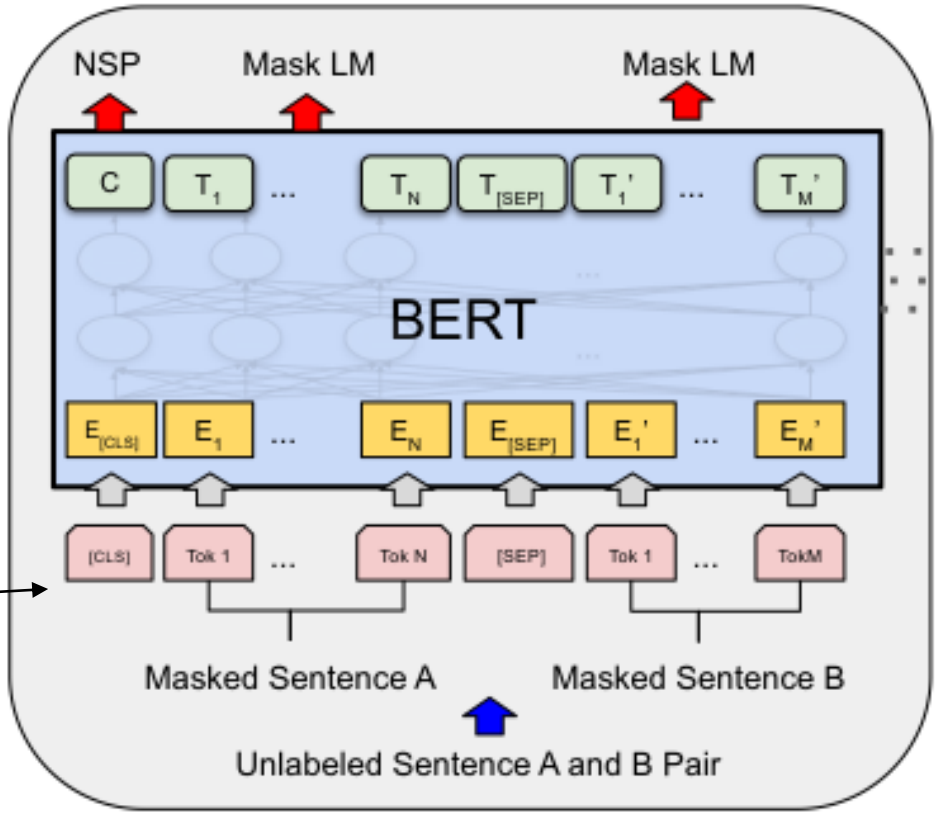
In a Transformer encoder



BERT structure

Devlin et al., 2018

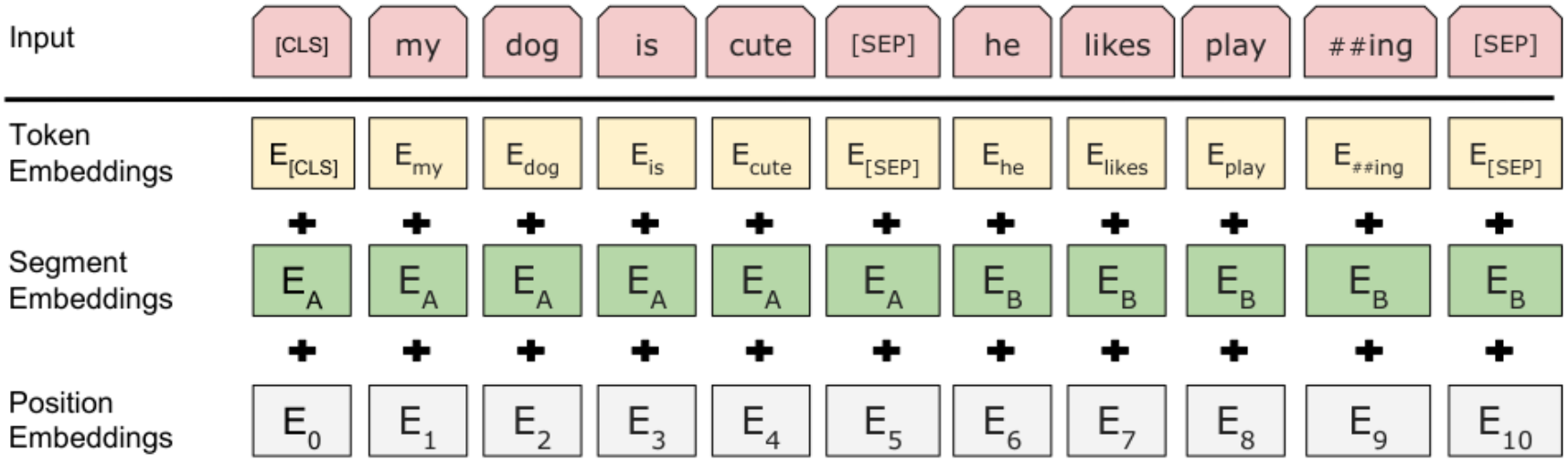
Transformer encoder



Some tokens are masked randomly

Input tokens: ①words ② [CLS] ③[SEP] ④ [MASK]

Input representation



- All embedding vectors have length 768
- All embedding vectors are learnable
- Segment embeddings have just 2 vectors of length 768
  - One for sentence A and the other for sentence B

Pre-training tasks

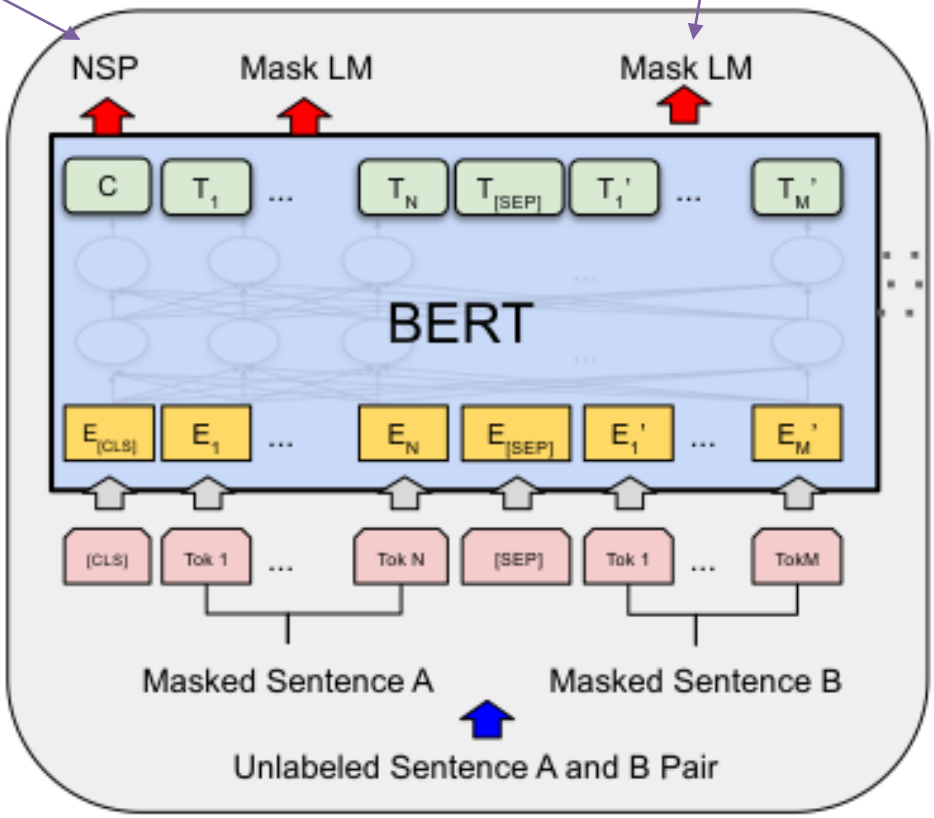
Devlin et al., 2018

Task 2: NSP

B is the next sentence to A?

Task 1: Masked LM:

What are the masked tokens?

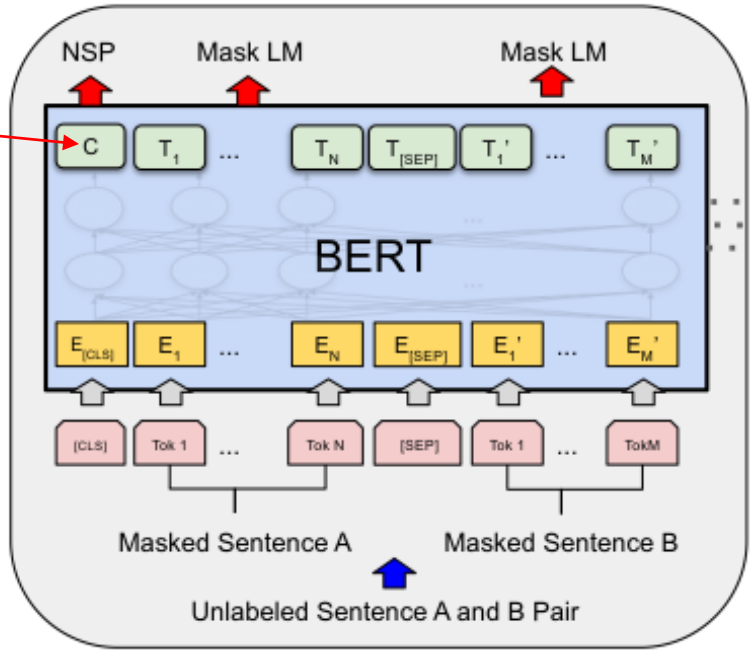


Fine-tuning example 1

- MNLI: Given a pair of sentences, predict whether the second sentence is an entailment, contradiction, or neutral with respect to the first one

Add an FC layer to C and apply softmax for classification

Input two sentences

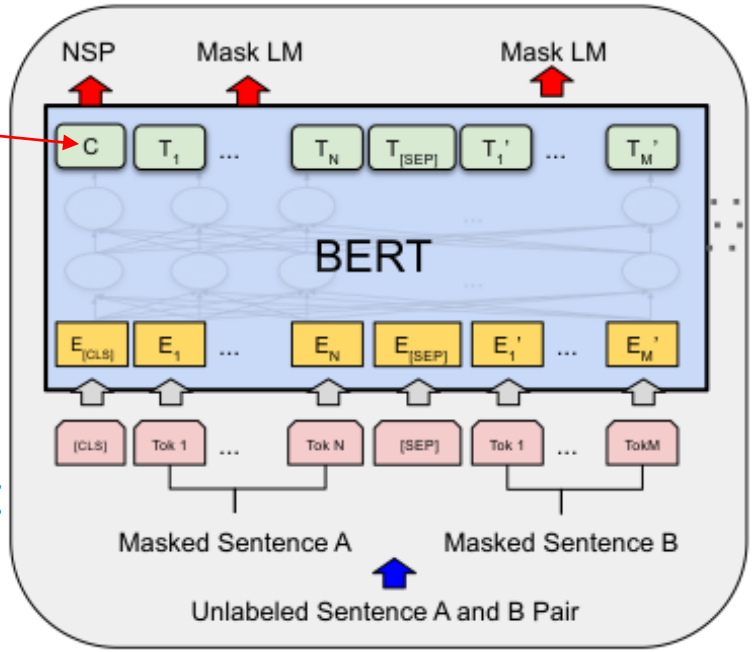


Fine-tuning example 2

- SST-2: a binary single-sentence classification task (sentiment for movie reviews)

Add an FC layer to C and apply softmax for classification

Input the single sentence plus a [SEP] at the end

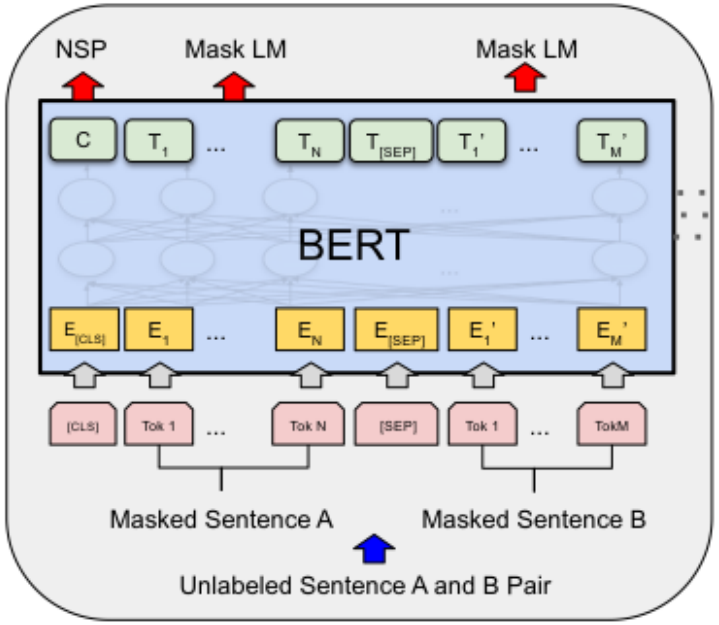


Summary of Part III

Pre-training → Fine-tuning

(On a large dataset)

Transformer encoder



Outline

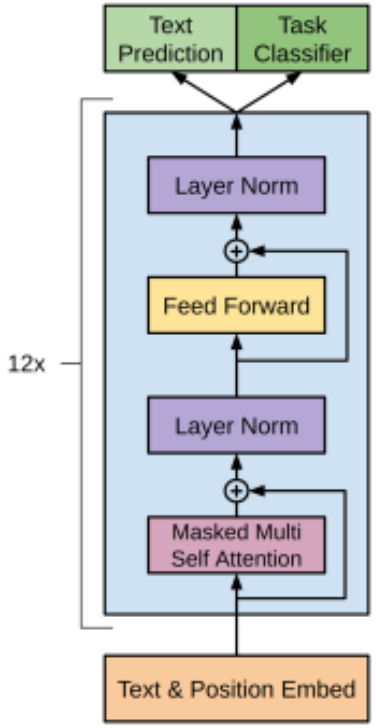
I. Self-attention
II. Transformer
III. BERT
IV. GPT
V. Summary

Generative Pre-Training (GPT1)

Radford et al. 2018

Pre-training → Fine-tuning

(On a large dataset)



- Model
  - A Transformer decoder w/o cross-attention
- Learning: $\max L_1(u) + \lambda L_2(C)$
  - Unsupervised pre-training

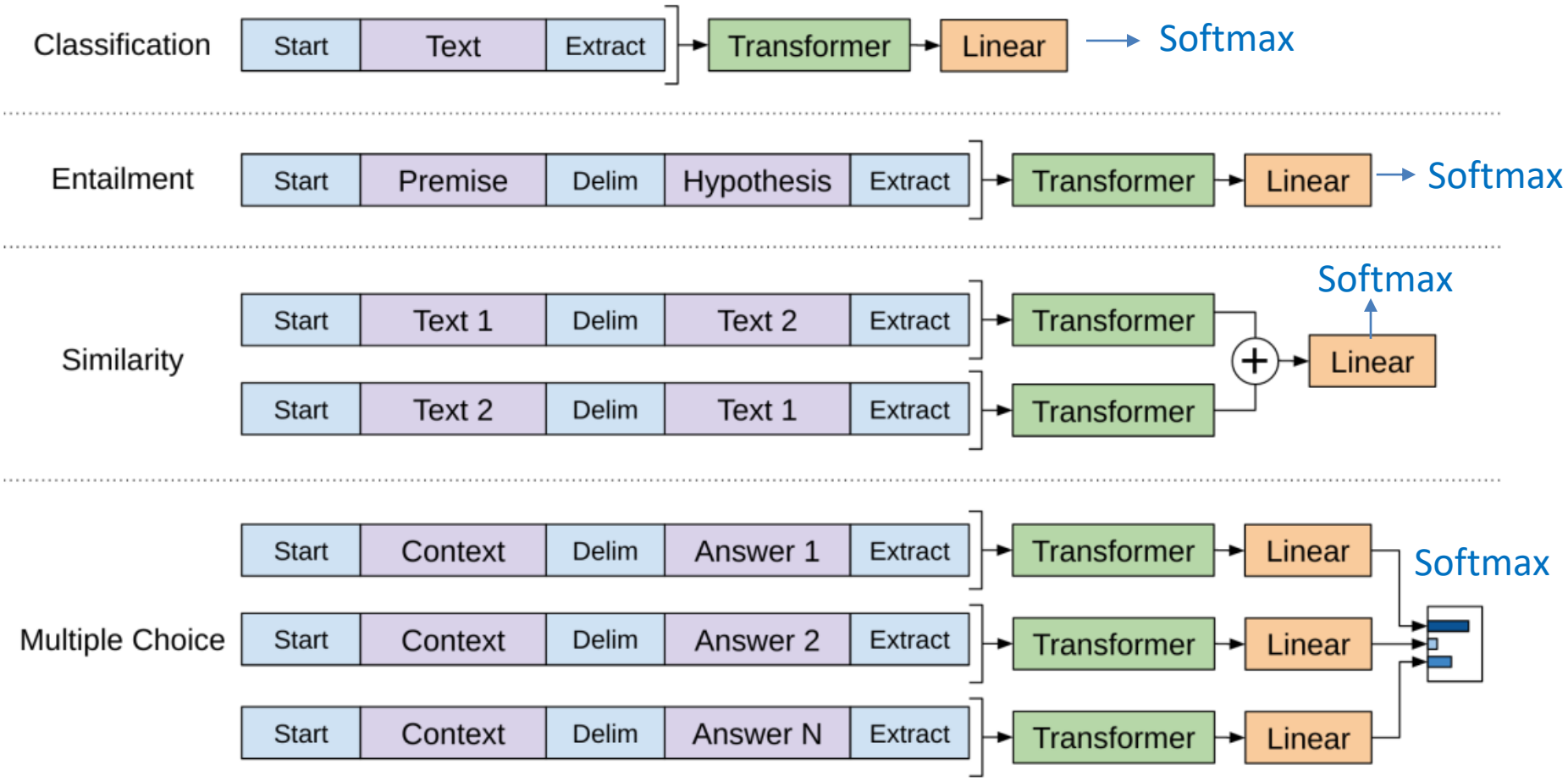
$$L_1(\mathcal{U}) = \sum_i \log P(u_i | u_{i-k}, \ldots, u_{i-1}; \Theta)$$

  - Supervised fine-tuning

$$L_2(\mathcal{C}) = \sum_{(x,y)} \log P(y | x^1, \ldots, x^m).$$

label

For different tasks



GPT2

Radford et al. 2019

- Model
  - GPT1 plus a few modifications
- Input
  - Word-level representation is restricted to represent certain strings
  - UTF-8 bytes representation didn't show good performance
  - A method in-between: Byte Pair Encoding (BPE)
- Training dataset
  - We scraped all outbound links from Reddit, which received at least 3 karma
  - WebText: contains the text subset of these 45 million link

High quality

GPT3

Brown et al. 2020

The basic pre-training approach, including model, data, and training, is similar to the process of GPT2, with relatively straightforward scaling up of the model size, dataset size and diversity, and length of training

Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}	d_{head}	Batch Size	Learning Rate
GPT-3 Small	125M	12	768	12	64	0.5M	6.0×10^{-4}
GPT-3 Medium	350M	24	1024	16	64	0.5M	3.0×10^{-4}
GPT-3 Large	760M	24	1536	16	96	0.5M	2.5×10^{-4}
GPT-3 XL	1.3B	24	2048	24	128	1M	2.0×10^{-4}
GPT-3 2.7B	2.7B	32	2560	32	80	1M	1.6×10^{-4}
GPT-3 6.7B	6.7B	32	4096	32	128	2M	1.2×10^{-4}
GPT-3 13B	13.0B	40	5140	40	128	2M	1.0×10^{-4}
GPT-3 175B or “GPT-3”	175.0B	96	12288	96	128	3.2M	0.6×10^{-4}

The training dataset

A combination of these datasets

Dataset	Quantity (tokens)	Weight in training mix	Epochs elapsed when training for 300B tokens
Common Crawl (filtered)	410 billion	60%	0.44
WebText2	19 billion	22%	2.9
Books1	12 billion	8%	1.9
Books2	55 billion	8%	0.43
Wikipedia	3 billion	3%	3.4

Higher-quality datasets such as WebText2 are sampled more frequently

Learning settings

Traditional fine-tuning is not used

Fine-tuning

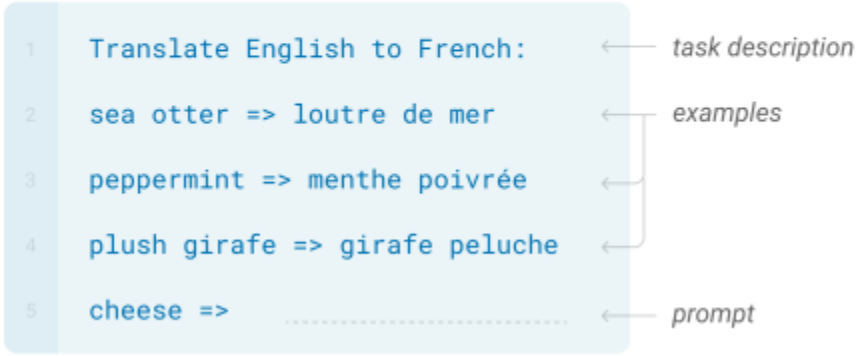
The model is trained via repeated gradient updates using a large corpus of example tasks.



In-context learning in GPT3

Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.



One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.



Learning settings

Traditional fine-tuning is not used

Fine-tuning

The model is trained via repeated gradient updates using a large corpus of example tasks.



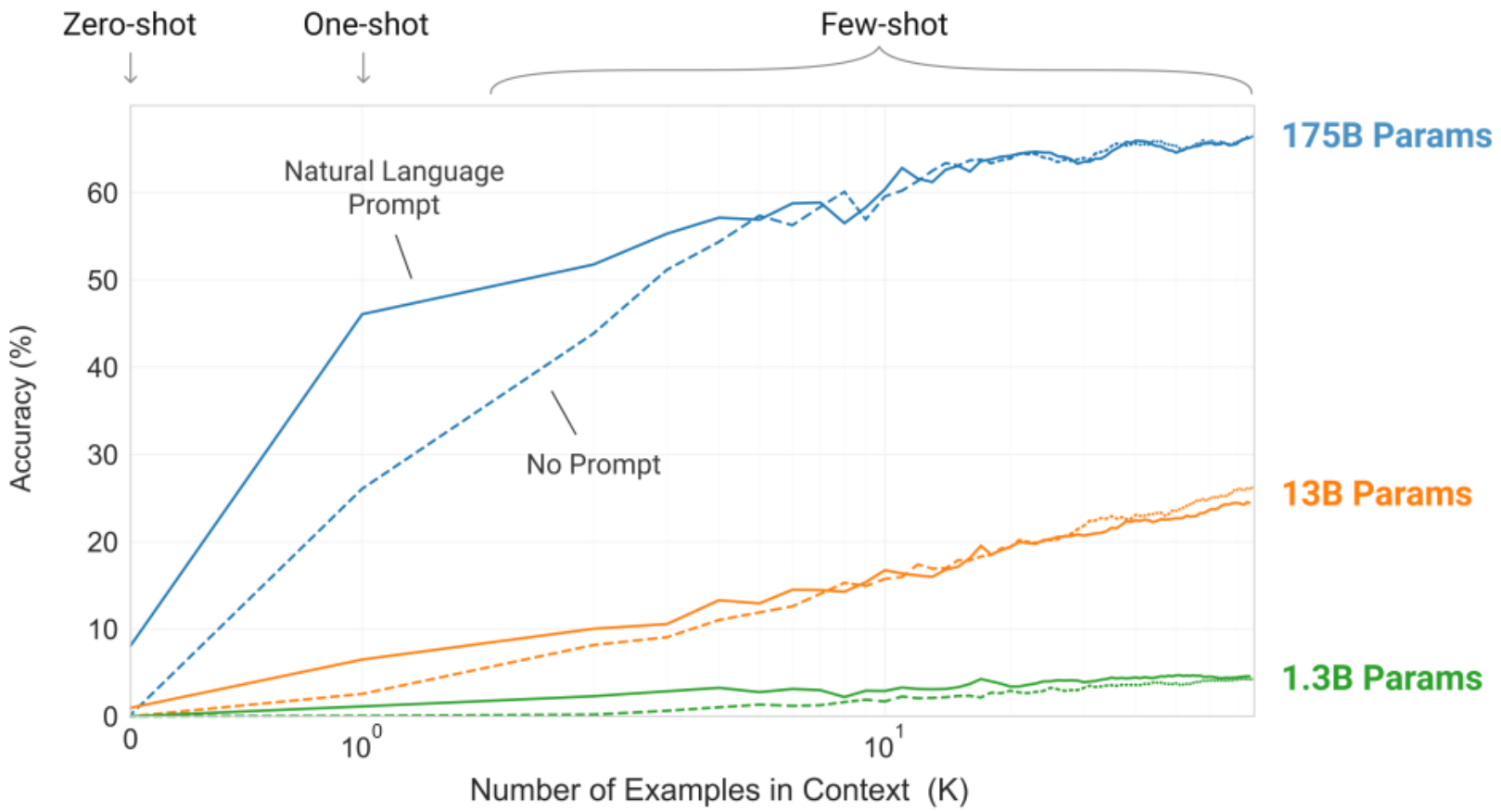
In-context learning in GPT3

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.



Larger models need less in-context information



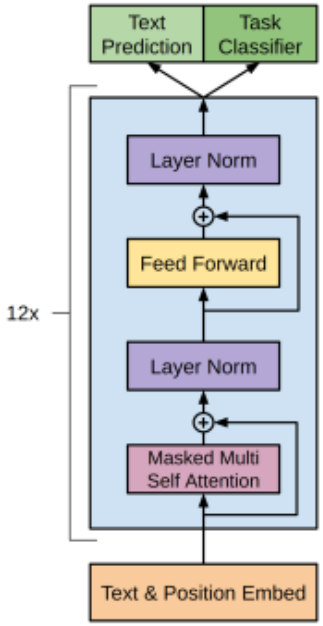
On a simple task requiring the model to remove random symbols from a word (see the original paper for details)

Summary of Part IV

Pre-training → Fine-tuning

(On a large dataset)

Transformer decoder



- GPT1
- GPT2
- GPT3
- ChatGPT
- GPT4

多选题 1分

设置

Which claims are true?

A BERT uses Transformer encoder only

B BERT uses Transformer decoder only

C GPT uses Transformer decoder only

D GPT uses both encoder and decoder

提交

单选题 1分

设置

Does GPT3 use the finetuning strategy?

A Yes

B No

提交

Outline

I. Self-attention
II. Transformer
III. BERT
IV. GPT
V. Summary

Summary of this lecture

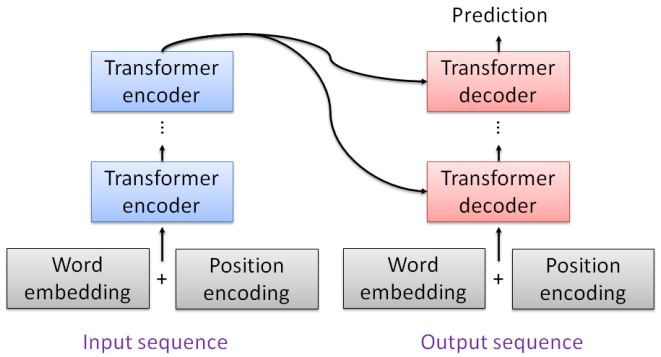
Knowledge

I. Self-attention

$$\boldsymbol{a}_t = \sum_{i=1}^{N} \text{softmax}\left(e_i^t(\boldsymbol{k}_i, \boldsymbol{q}_t)\right) \boldsymbol{v}_i$$

key | query | value

II. Transformer



Self-attention + Cross-attention

Encoder + Decoder

III. BERT

Transformer Encoder

III. GPT

Transformer Decoder

Main References

- Vaswani, Shazeer et al. (2017)
  Attention Is All You Need
  NeurIPS
- Devlin, Chang, Lee, & Toutanova (2018)
  BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding
  NAACL-HLT 2019
- Radford, Narasimhan, Salimans, & Sutskever (2018)
  Improving language understanding by generative pre-training
- Radford, et al. (2019)
  Language models are unsupervised multitask learners
  OpenAI blog 1.8
- Brown, Tom, et al. (2020)
  Language models are few-shot learners
  NeurIPS